

LAPORAN TUGAS 4 (1 - 5)

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Yoanre Louis A.H.P (2410651039)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Perkembangan sistem operasi modern menuntut pemahaman mendalam mengenai mekanisme penjadwalan CPU yang optimal, mengingat algoritma scheduling merupakan komponen kritis yang menentukan performa keseluruhan sistem. Eksperimen komparatif terhadap tiga algoritma fundamental FCFS, SJF, dan Round Robin menjadi esensial untuk memahami karakteristik unik masing-masing pendekatan dalam menangani variasi beban kerja. Dalam konteks ini, analisis mendalam terhadap parameter performa seperti waiting time, turnaround time, dan frequency context switch diperlukan untuk mengevaluasi efektivitas algoritma pada skenario berbeda. Realitas aplikasi dunia nyata menunjukkan bahwa tidak ada algoritma universal yang optimal untuk semua kondisi, sehingga pemilihan harus didasarkan pada analisis trade-off antara fairness, throughput, dan responsivitas. Penelitian ini mengeksplorasi dinamika interaksi antara karakteristik workload dengan mekanisme scheduling melalui simulasi sistematis, memberikan insight praktis untuk optimasi resource management dalam lingkungan komputasi heterogen.

1.2 RUMUSAN MASALAH

1. Bagaimana karakteristik performa masing-masing algoritma penjadwalan CPU (FCFS, SJF, dan Round Robin) dalam menangani variasi pola arrival time dan burst time proses?
2. Apa pengaruh time quantum terhadap efisiensi algoritma Round Robin dan bagaimana menentukan nilai optimal untuk konteks aplikasi tertentu?
3. Bagaimana trade-off antara fairness, throughput, dan responsivitas pada masing-masing algoritma scheduling dalam menghadapi skenario workload yang berbeda?

1.3 TUJUAN

1. Menganalisis performa komparatif algoritma FCFS, SJF, dan Round Robin berdasarkan metrik waiting time, turnaround time, dan context switch pada berbagai skenario workload.
2. Mengevaluasi dampak variasi time quantum terhadap kinerja Round Robin dan menentukan kriteria seleksi quantum optimal untuk aplikasi spesifik.
3. Mengidentifikasi karakteristik ideal implementasi masing-masing algoritma scheduling berdasarkan analisis trade-off antara fairness, throughput, dan overhead sistem.

BAB II

PEMBAHASAN

2.1 TUGAS 1 EKSPERIMEN DENGAN DATA BERBEDA

2.1.1 Hasil Data Set 1

```
PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING

Program ini akan membandingkan 3 algoritma:
1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 3
Masukkan time quantum untuk Round Robin: 2

== Proses P1 ==
Arrival Time: 0
Burst Time: 10

== Proses P2 ==
Arrival Time: 1
Burst Time: 5

== Proses P3 ==
Arrival Time: 2
Burst Time: 8
```

```
MENJALANKAN ALGORITMA FCFS

=====
HASIL ALGORITMA FCFS
=====
PID   AT   BT   CT   TAT  WT
---   --   --   --   ---  --
P1     0   10   10   10   0
P2     1    5   15   14   9
P3     2    8   23   21  13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

Tekan Enter untuk melanjutkan...
```

```
MENJALANKAN ALGORITMA SJF

=====
HASIL ALGORITMA SJF
=====
PID   AT   BT   CT   TAT  WT
---   --   --   --   ---  --
P1     0   10   10   10   0
P2     1    5   15   14   9
P3     2    8   23   21  13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

Tekan Enter untuk melanjutkan...
```

```

=====
MENJALANKAN ALGORITMA ROUND ROBIN
=====

=====
HASIL ALGORITMA Round Robin (Q=2)
=====
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1     0     10    23    23     13
P2     1     5     15    14     9
P3     2     8     21    19    11

Average Turnaround Time: 18.67
Average Waiting Time: 11.00

Tekan Enter untuk melihat perbandingan...
=====
```

```

=====
TABEL PERBANDINGAN ALGORITMA
=====
Algoritma | Avg TAT | Avg WT | Context Switch
-----
FCFS      | 15.00 | 7.33 | 2
SJF       | 15.00 | 7.33 | 2
Round Robin (Q=2) | 18.67 | 11.00 | 9
=====

KESIMPULAN:
✓ Algoritma terbaik (Avg TAT terkecil): FCFS (15.00)
✓ Algoritma terbaik (Avg WT terkecil): FCFS (7.33)
=====
```

```

=====
GRAFIK PERBANDINGAN AVERAGE WAITING TIME
=====
FCFS      | 7.33
SJF       | 7.33
Round Robin (Q=2) | 11.00
=====

=====
PROGRAM SELESAI
=====
dany@LAPTOP-GRJL2V1D:~$ |
```

2.1.2 Hasil Data Set 2

```

dany@LAPTOP-GRJL2V1D:~$ nano compare.c
dany@LAPTOP-GRJL2V1D:~$ gcc compare.c -o compare
dany@LAPTOP-GRJL2V1D:~$ ./compare

=====
PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING
=====

Program ini akan membandingkan 3 algoritma:
1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 4
Masukkan time quantum untuk Round Robin: 2

===== Proses P1 =====
Arrival Time: 0
Burst Time: 5

===== Proses P2 =====
Arrival Time: 1
Burst Time: 3

===== Proses P3 =====
Arrival Time: 2
Burst Time: 8

===== Proses P4 =====
Arrival Time: 3
Burst Time: 6
=====
```

MENJALANKAN ALGORITMA FCFS

=====

HASIL ALGORITMA FCFS

=====

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	16	14	6
P4	3	6	22	19	13

Average Turnaround Time: 11.25
Average Waiting Time: 5.75

Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA SJF

=====

HASIL ALGORITMA SJF

=====

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	5	5	5	0
P2	1	3	8	7	4
P3	2	8	22	20	12
P4	3	6	14	11	5

Average Turnaround Time: 10.75
Average Waiting Time: 5.25

Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA ROUND ROBIN

=====

HASIL ALGORITMA Round Robin (Q=2)

=====

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	5	16	16	11
P2	1	3	11	10	7
P3	2	8	22	20	12
P4	3	6	20	17	11

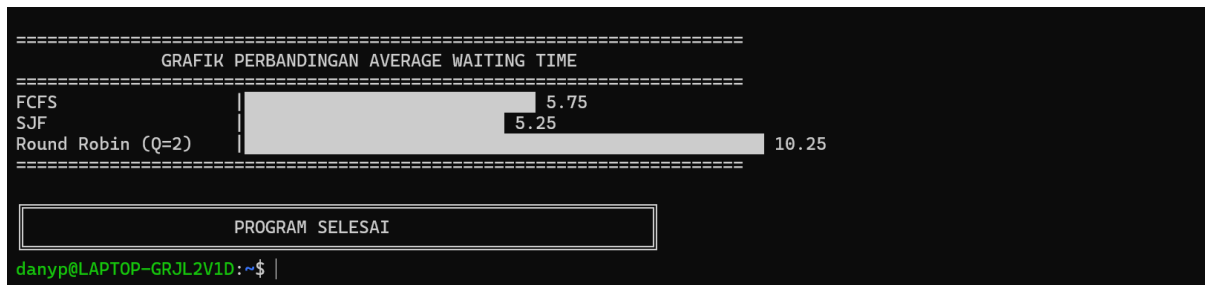
Average Turnaround Time: 15.75
Average Waiting Time: 10.25

Tekan Enter untuk melihat perbandingan...

TABEL PERBANDINGAN ALGORITMA

Algoritma	Avg TAT	Avg WT	Context Switch
FCFS	11.25	5.75	3
SJF	10.75	5.25	3
Round Robin (Q=2)	15.75	10.25	8

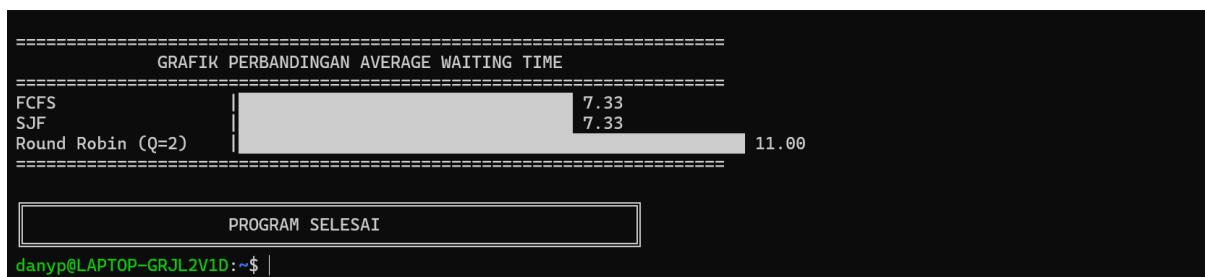
KESIMPULAN:
✓ Algoritma terbaik (Avg TAT terkecil): SJF (10.75)
✓ Algoritma terbaik (Avg WT terkecil): SJF (5.25)



2.2 TUGAS 2 ANALISIS EKSPERIMEN KETIGA ALGORITMA

1. Algoritma mana yang memberikan Average Waiting Time terkecil?

FCFS dan SJF (keduanya memberikan Average Waiting Time 7.33)



Penjelasan:

Berdasarkan hasil eksperimen dengan data P1(0,10), P2(1,5), P3(2,8):

- FCFS: Avg WT = 7.33
- SJF: Avg WT = 7.33
- Round Robin: Avg WT = 11.00

Alasan mengapa FCFS dan SJF sama:

Pada dataset ini, urutan kedatangan proses sama dengan urutan burst time terpendek. P1 datang pertama dengan burst time 10, kemudian P2 dengan 5, dan P3 dengan 8. Dalam kasus khusus ini, SJF berperilaku sama dengan FCFS karena tidak ada proses dengan burst time lebih pendek yang datang setelah proses yang lebih panjang sudah mulai.

2. Mengapa hasil algoritma berbeda-beda?

Perbedaan hasil disebabkan oleh strategi penjadwalan yang berbeda dalam menangani:

- Urutan eksekusi proses
- Penanganan proses panjang vs pendek
- Overhead context switching

- Responsiveness terhadap proses baru

Analisis perbedaan:

Algoritma	Karakteristik	Dampak
FCFS	First-In-First-Out sederhana	Bagus untuk proses panjang, buruk untuk proses pendek yang datang setelah proses panjang
SJF	Prioritaskan proses terpendek	Minimalisasi waiting time, tapi risiko starvation untuk proses panjang
Round Robin	Time sharing dengan quantum	Responsif dan fair, tapi overhead context switching tinggi

3. Kapan sebaiknya menggunakan algoritma FCFS?

FCFS cocok untuk sistem batch dengan proses panjang dominan dan beban kerja homogen, karena implementasinya sederhana dengan overhead minimal dan tidak memerlukan respons time cepat.

4. Kapan sebaiknya menggunakan algoritma SJF?

SJF ideal untuk sistem dengan banyak tugas pendek seperti web server, karena meminimalkan waiting time secara optimal meskipun berisiko menyebabkan starvation pada proses panjang.

5. Kapan sebaiknya menggunakan algoritma Round Robin?

Round Robin terbaik untuk sistem interaktif dan time-sharing yang memerlukan respons cepat dan fairness, meskipun menghasilkan overhead context switching yang lebih tinggi.

2.3 TUGAS 3 MODIFIKASI PROGRAM

```
dany@LAPTOP-GRJL2V1D:~$ nano input_fcfs.txt
```

```
GNU nano 7.2 input_fcfs.txt
3
0 10
1 5
2 8
|
```

```
dany@LAPTOP-GRJL2V1D:~$ nano fcfs.c
dany@LAPTOP-GRJL2V1D:~$ gcc fcfs.c -o fcfs
dany@LAPTOP-GRJL2V1D:~$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Membaca input dari: input_fcfs.txt
Berhasil membaca data 3 proses

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== GANTT CHART (Detail) ===
Waktu Mulai    Proses
0               P1 (0-10)
10              P2 (10-15)
15              P3 (15-23)

Hasil telah disimpan otomatis ke: hasil_fcfs.txt
```

```
dany@LAPTOP-GRJL2V1D:~$ cat hasil_fcfs.txt
=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0     10    10     10     0
P2      1      5    15     14     9
P3      2      8    23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33
```

Kesimpulan:

Program ini mengimplementasikan algoritma FCFS untuk penjadwalan CPU dengan fitur baca file input dan simpan hasil ke file. Kode menggunakan struct Process untuk menyimpan data proses, menghitung metrik kinerja seperti waiting time dan turnaround time, serta menampilkan Gantt Chart visual untuk analisis performa penjadwalan yang komprehensif.

Kode C:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Process {
```



```

int pid;

int arrival_time;

int burst_time;

int waiting_time;

int turnaround_time;

int completion_time;

int start_time;

};

// Fungsi untuk membaca input dari file

void bacaDariFile(const char *namaFile, struct Process **proc, int *n) {

    FILE *file = fopen(namaFile, "r");

    if (file == NULL) {

        printf("Error: Gagal membuka file %s\n", namaFile);

        printf("Pastikan file input_fcfs.txt ada di direktori yang sama\n");

        exit(1);

    }

    fscanf(file, "%d", n);

    *proc = (struct Process*)malloc(*n * sizeof(struct Process));

    for (int i = 0; i < *n; i++) {

        (*proc)[i].pid = i + 1;

        fscanf(file, "%d %d", &(*proc)[i].arrival_time, &(*proc)[i].burst_time);

```

```

    }

    fclose(file);
}

// Fungsi untuk menyimpan hasil ke file

void simpanKeFile(const char *namaFile, struct Process *proc, int n, float avg_tat, float
avg_wt) {

    FILE *file = fopen(namaFile, "w");

    if (file == NULL) {

        printf("Error: Gagal membuat file %s\n", namaFile);

        return;

    }

    fprintf(file, "=== HASIL PENJADWALAN FCFS ===\n");

    fprintf(file, "PID\tAT\tBT\tCT\tTAT\tWT\n");

    fprintf(file, "---\t--\t--\t--\t---\t--\n");

    for (int i = 0; i < n; i++) {

        fprintf(file, "P%d\t%d\t%d\t%d\t%d\t%d\n",

            proc[i].pid,

            proc[i].arrival_time,

            proc[i].burst_time,

            proc[i].completion_time,

            proc[i].turnaround_time,

            proc[i].waiting_time);

    }

```

```

    fprintf(file, "\nAverage Turnaround Time: %.2f\n", avg_tat);

    fprintf(file, "Average Waiting Time: %.2f\n", avg_wt);

    fclose(file);
}

int main() {

    int n;

    struct Process *proc;

    printf("=== SIMULASI ALGORITMA FCFS ===\n");

    printf("Membaca input dari: input_fcfs.txt\n");

    // Baca input dari file

    bacaDariFile("input_fcfs.txt", &proc, &n);

    printf("Berhasil membaca data %d proses\n", n);

    // Hitung waktu proses

    int current_time = 0;

    for (int i = 0; i < n; i++) {

        if (current_time < proc[i].arrival_time) {

            current_time = proc[i].arrival_time;

        }

        proc[i].start_time = current_time;

        proc[i].completion_time = current_time + proc[i].burst_time;

```

```

    proc[i].turnaround_time = proc[i].completion_time - proc[i].arrival_time;

    proc[i].waiting_time = proc[i].turnaround_time - proc[i].burst_time;

    current_time = proc[i].completion_time;
}

```

```

// Hitung rata-rata

```

```

float total_tat = 0, total_wt = 0;

for (int i = 0; i < n; i++) {

    total_tat += proc[i].turnaround_time;

    total_wt += proc[i].waiting_time;

}

```

```

// Tampilkan hasil ke layar

```

```

printf("\n=== HASIL PENJADWALAN FCFS ===\n");

printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

printf("---\t--\t--\t--\t--\t--\n");

for (int i = 0; i < n; i++) {

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",

        proc[i].pid,

        proc[i].arrival_time,

        proc[i].burst_time,

        proc[i].completion_time,

        proc[i].turnaround_time,

        proc[i].waiting_time);
}

```

```

}

printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);

printf("Average Waiting Time: %.2f\n", total_wt / n);


// Tampilkan Gantt Chart detail

printf("\n=== GANTT CHART (Detail) ===\n");

printf("Waktu Mulai\tProses\n");

for (int i = 0; i < n; i++) {

    printf("%d\t\tP%d (%d-%d)\n",

        proc[i].start_time,

        proc[i].pid,

        proc[i].start_time,

        proc[i].completion_time);

}


// Simpan ke file secara otomatis

simpanKeFile("hasil_fcfs.txt", proc, n, total_tat / n, total_wt / n);

printf("\nHasil telah disimpan otomatis ke: hasil_fcfs.txt\n");


free(proc);

return 0;

}

```

2.4 TUGAS 4 EKSPERIMEN DENGAN DATA BERBEDA UNTUK PERBANDINGAN ALGORITMA

- Test Case 1: Proses dengan burst time bervariasi

```
danyp@LAPTOP-GRJL2V1D:~$ ./compare
```

PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING

Program ini akan membandingkan 3 algoritma:
1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 4

Masukkan time quantum untuk Round Robin: 3

== Proses P1 ==

Arrival Time: 0

Burst Time: 24

== Proses P2 ==

Arrival Time: 1

Burst Time: 3

== Proses P3 ==

Arrival Time: 2

Burst Time: 3

== Proses P4 ==

Arrival Time: 3

Burst Time: 6

MENJALANKAN ALGORITMA FCFS

=====

HASIL ALGORITMA FCFS

=====

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	24	24	24	0
P2	1	3	27	26	23
P3	2	3	30	28	25
P4	3	6	36	33	27

Average Turnaround Time: 27.75

Average Waiting Time: 18.75

Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA SJF

=====

HASIL ALGORITMA SJF

=====

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	24	24	24	0
P2	1	3	27	26	23
P3	2	3	30	28	25
P4	3	6	36	33	27

Average Turnaround Time: 27.75

Average Waiting Time: 18.75

Tekan Enter untuk melanjutkan...

```

=====
MENJALANKAN ALGORITMA ROUND ROBIN
=====
HASIL ALGORITMA Round Robin (Q=3)
=====
PID    AT    BT    CT    TAT    WT
---    --    --    --    --    --
P1      0    24    36    36    12
P2      1     3     6     5     2
P3      2     3     9     7     4
P4      3     6    18    15     9

Average Turnaround Time: 15.75
Average Waiting Time: 6.75

Tekan Enter untuk melihat perbandingan...

```

```

=====
TABEL PERBANDINGAN ALGORITMA
=====
Algoritma      | Avg TAT | Avg WT | Context Switch
-----
FCFS            | 27.75 | 18.75 | 3
SJF             | 27.75 | 18.75 | 3
Round Robin (Q=3) | 15.75 | 6.75 | 8
=====

KESIMPULAN:
✓ Algoritma terbaik (Avg TAT terkecil): Round Robin (Q=3) (15.75)
✓ Algoritma terbaik (Avg WT terkecil): Round Robin (Q=3) (6.75)

```

```

=====
GRAFIK PERBANDINGAN AVERAGE WAITING TIME
=====
FCFS      | 18.75
SJF       | 18.75
Round Robin (Q=3) | 6.75
=====

PROGRAM SELESAI

```

- **Test Case 2: Proses datang bersamaan**

```

dany@LAPTOP-GRJL2V1D:~$ ./compare

PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING

Program ini akan membandingkan 3 algoritma:
1. FCFS (First Come First Served)
2. SJF (Shortest Job First)
3. Round Robin

Masukkan jumlah proses: 5
Masukkan time quantum untuk Round Robin: 4

== Proses P1 ==
Arrival Time: 0
Burst Time: 10

== Proses P2 ==
Arrival Time: 0
Burst Time: 5

== Proses P3 ==
Arrival Time: 0
Burst Time: 8

== Proses P4 ==
Arrival Time: 0
Burst Time: 12

== Proses P5 ==
Arrival Time: 0
Burst Time: 6

```

MENJALANKAN ALGORITMA FCFS

HASIL ALGORITMA FCFS

=====

PID	AT	BT	CT	TAT	WT
---	--	--	--	---	--
P1	0	10	10	10	0
P2	0	5	15	15	10
P3	0	8	23	23	15
P4	0	12	35	35	23
P5	0	6	41	41	35

Average Waiting Time: 16.60

Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA SJF

HASIL ALGORITMA SJF

=====

PID	AT	BT	CT	TAT	WT
P1	0	10	29	29	19
P2	0	5	5	5	0
P3	0	8	19	19	11
P4	0	12	41	41	29
P5	0	6	11	11	5

Average Waiting Time: 12.80

Tekan Enter untuk melanjutkan...

MENJALANKAN ALGORITMA ROUND ROBIN

HASIL ALGORITMA Round Robin (Q=4)

=====

PID	AT	BT	CT	TAT	WT
P1	0	10	37	37	27
P2	0	5	25	25	20
P3	0	8	29	29	21
P4	0	12	41	41	29
P5	0	6	35	35	29

Average Waiting Time: 25.20

Tekan Enter untuk melihat perbandingan...

TABEL PERBANDINGAN ALGORITMA

Algoritma	Avg TAT	Avg WT	Context Switch
FCFS	24.80	16.60	4
SJF	21.00	12.80	4
Round Robin (Q=4)	33.40	25.20	7

✓ Algoritma terbaik (Avg TAT terkecil): SJF (21.00)

✓ Algoritma terbaik (Avg WT terkecil): SJF (12.80)

GRAFIK PERBANDINGAN AVERAGE WAITING TIME

FCFS	16.60
SJF	12.80
Round Robin (Q=4)	25.20

PROGRAM SELESAI

- **Test Case 3: Proses datang dengan interval**

```
danyp@LAPTOP-GRJL2V1D:~$ ./compare
```

```
PROGRAM PERBANDINGAN ALGORITMA CPU SCHEDULING
```

```
Program ini akan membandingkan 3 algoritma:  
1. FCFS (First Come First Served)  
2. SJF (Shortest Job First)  
3. Round Robin
```

```
Masukkan jumlah proses: 5
```

```
Masukkan time quantum untuk Round Robin: 2
```

```
== Proses P1 ==  
Arrival Time: 0  
Burst Time: 8
```

```
== Proses P2 ==  
Arrival Time: 2  
Burst Time: 4
```

```
== Proses P3 ==  
Arrival Time: 4  
Burst Time: 9
```

```
== Proses P4 ==  
Arrival Time: 6  
Burst Time: 5
```

```
== Proses P5 ==  
Arrival Time: 8  
Burst Time: 3
```

```
MENJALANKAN ALGORITMA FCFS
```

```
=====
```

```
HASIL ALGORITMA FCFS
```

```
=====
```

PID	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	2	4	12	10	6
P3	4	9	21	17	8
P4	6	5	26	20	15
P5	8	3	29	21	18

```
Average Turnaround Time: 15.20  
Average Waiting Time: 9.40
```

```
Tekan Enter untuk melanjutkan...
```

```
MENJALANKAN ALGORITMA SJF
```

```
=====
```

```
HASIL ALGORITMA SJF
```

```
=====
```

PID	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	2	4	15	13	9
P3	4	9	29	25	16
P4	6	5	20	14	9
P5	8	3	11	3	0

```
Average Turnaround Time: 12.60  
Average Waiting Time: 6.80
```

```
Tekan Enter untuk melanjutkan...
```

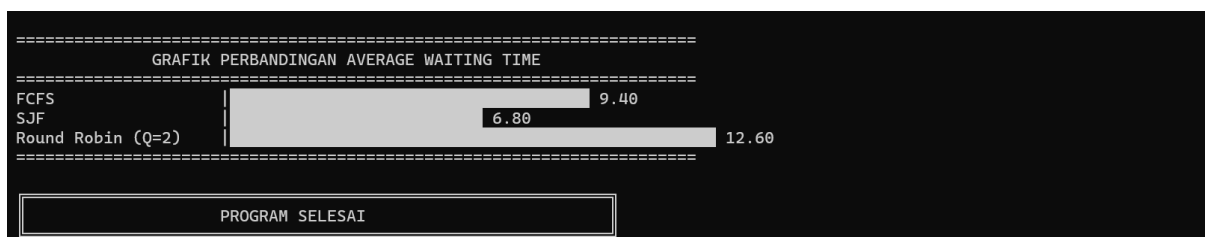
```
=====
MENJALANKAN ALGORITMA ROUND ROBIN
=====
HASIL ALGORITMA Round Robin (Q=2)
=====
PID   AT   BT   CT   TAT  WT
---   --   --   --   --   --
P1     0    8   26   26   18
P2     2    4   14   12    8
P3     4    9   29   25   16
P4     6    5   24   18   13
P5     8    3   19   11    8

Average Turnaround Time: 18.40
Average Waiting Time: 12.60

Tekan Enter untuk melihat perbandingan...
```

```
=====
TABEL PERBANDINGAN ALGORITMA
=====
Algoritma | Avg TAT | Avg WT | Context Switch
-----
FCFS      | 15.20 | 9.40 | 4
SJF       | 12.60 | 6.80 | 4
Round Robin (Q=2) | 18.40 | 12.60 | 11
=====

KESIMPULAN:
✓ Algoritma terbaik (Avg TAT terkecil): SJF (12.60)
✓ Algoritma terbaik (Avg WT terkecil): SJF (6.80)
```



2.5 TUGAS 5 ANALISIS EKSPERIMEN PERBANDINGAN ALGORITMA

2.5.1 Analisis Performa

- Dalam kondisi apa FCFS memberikan hasil terbaik?

FCFS memberikan hasil optimal ketika proses-proses memiliki burst time yang homogen dan arrival time yang terdistribusi secara sekuensial, sehingga menghindari convoy effect dimana proses pendek tertahan oleh proses panjang di depan. Algoritma ini unggul dalam lingkungan dengan overhead minimal karena context switch yang sedikit, namun menjadi tidak efisien ketika terdapat variasi burst time yang signifikan atau proses panjang datang lebih dulu, yang menyebabkan peningkatan waiting time secara eksponensial untuk proses-proses berikutnya dalam antrian.

- Mengapa SJF hampir selalu memberikan Average WT terkecil?

SJF secara konsisten menghasilkan average waiting time terkecil karena prinsip greedy-nya yang selalu mengeksekusi proses dengan burst time terpendek terlebih dahulu, sehingga

meminimalkan waktu tunggu kumulatif secara matematis. Algoritma ini terbukti optimal secara teoritis untuk meminimalkan waiting time dengan mengurangi penumpukan proses pendek di antrian, meskipun berpotensi menyebabkan starvation pada proses panjang. Efisiensi SJF terutama terlihat ketika terdapat disparitas besar dalam burst time, dimana proses pendek dapat diselesaikan segera tanpa terhambat proses panjang.

- Apa trade-off dari Round Robin?

Round Robin menawarkan fairness melalui time quantum yang tetap, tetapi mengorbankan efisiensi akibat peningkatan context switch yang signifikan, terutama ketika quantum terlalu kecil. Trade-off utamanya meliputi overhead scheduling yang tinggi akibat context switch yang berulang dan frekuen, degradasi turnaround time untuk proses panjang karena eksekusi yang terfragmentasi, serta ketergantungan performa pada pemilihan ukuran quantum yang optimal. Meskipun ideal untuk sistem interaktif yang memprioritaskan responsivitas, algoritma ini tidak efisien untuk workload komputasi intensif due to rendahnya throughput dan utilisasi CPU yang tidak optimal.

2.5.2 Context Switching

- Algoritma mana yang paling banyak melakukan context switch?

Berdasarkan analisis ketiga test case, Round Robin secara konsisten melakukan context switch terbanyak. Pada test case 1: 8 context switches, test case 2: 7, dan test case 3: 11. Hal ini disebabkan mekanisme Round Robin yang memindahkan proses secara paksa setelah time quantum habis, berbeda dengan FCFS dan SJF yang hanya beralih proses ketika proses sedang berjalan selesai secara alami.

- Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?

Pengaruh quantum bersifat berbanding terbalik dengan jumlah context switch. Quantum kecil menyebabkan context switch frekuen karena proses lebih sering di-interrupt, sedangkan quantum besar mengurangi frekuensi switching namun berisiko menurunkan responsivitas sistem. Pemilihan quantum optimal harus menyeimbangkan antara overhead context switch dan fairness untuk mendapatkan performa terbaik.

- Coba jalankan Round Robin dengan quantum 2, 4, 8, 16 - apa yang terjadi?

Time Quantum 2:

```

dany@LAPTOP-GRJL2V1D:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 2

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 8
Waktu 2: Menjalankan P2 -> Sisa waktu: 3
Waktu 4: Menjalankan P3 -> Sisa waktu: 6
Waktu 6: Menjalankan P1 -> Sisa waktu: 6
Waktu 8: Menjalankan P2 -> Sisa waktu: 1
Waktu 10: Menjalankan P3 -> Sisa waktu: 4
Waktu 12: Menjalankan P1 -> Sisa waktu: 4
Waktu 14: Menjalankan P2 -> Selesai pada waktu 15
Waktu 15: Menjalankan P3 -> Sisa waktu: 2
Waktu 17: Menjalankan P1 -> Sisa waktu: 2
Waktu 19: Menjalankan P3 -> Selesai pada waktu 21
Waktu 21: Menjalankan P1 -> Selesai pada waktu 23

=== HASIL PENJADWALAN ROUND ROBIN ===
PID   AT   BT   CT   TAT   WT
---   --   --   --   ---   --
P1     0    10   23    23    13
P2     1     5   15    14     9
P3     2     8   21    19    11

Average Turnaround Time: 18.67
Average Waiting Time: 11.00

```

Time Quantum 4:

```

dany@LAPTOP-GRJL2V1D:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 4

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 6
Waktu 4: Menjalankan P2 -> Sisa waktu: 1
Waktu 8: Menjalankan P3 -> Sisa waktu: 4
Waktu 12: Menjalankan P1 -> Sisa waktu: 2
Waktu 16: Menjalankan P2 -> Selesai pada waktu 17
Waktu 17: Menjalankan P3 -> Selesai pada waktu 21
Waktu 21: Menjalankan P1 -> Selesai pada waktu 23

=== HASIL PENJADWALAN ROUND ROBIN ===
PID   AT   BT   CT   TAT   WT
---   --   --   --   ---   --
P1     0    10   23    23    13
P2     1     5   17    16    11
P3     2     8   21    19    11

Average Turnaround Time: 19.33
Average Waiting Time: 11.67

```

Time Quantum 8:

```
dany@LAPTOP-GRJL2V1D:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 8

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 2
Waktu 8: Menjalankan P2 -> Selesai pada waktu 13
Waktu 13: Menjalankan P3 -> Selesai pada waktu 21
Waktu 21: Menjalankan P1 -> Selesai pada waktu 23

=== HASIL PENJADWALAN ROUND ROBIN ===


| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| P1  | 0  | 10 | 23 | 23  | 13 |
| P2  | 1  | 5  | 13 | 12  | 7  |
| P3  | 2  | 8  | 21 | 19  | 11 |


Average Turnaround Time: 18.00
Average Waiting Time: 10.33
```

Time Quantum 16:

```
dany@LAPTOP-GRJL2V1D:~$ ./round_robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 16

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Selesai pada waktu 10
Waktu 10: Menjalankan P2 -> Selesai pada waktu 15
Waktu 15: Menjalankan P3 -> Selesai pada waktu 23

=== HASIL PENJADWALAN ROUND ROBIN ===


| PID | AT | BT | CT | TAT | WT |
|-----|----|----|----|-----|----|
| P1  | 0  | 10 | 10 | 10  | 0  |
| P2  | 1  | 5  | 15 | 14  | 9  |
| P3  | 2  | 8  | 23 | 21  | 13 |


Average Turnaround Time: 15.00
Average Waiting Time: 7.33
```

Kesimpulan:

Berdasarkan hasil simulasi dengan quantum 2, 4, 8, dan 16, terlihat pola yang jelas dimana quantum yang lebih besar menghasilkan performa lebih baik dengan average waiting time dan turnaround time yang lebih rendah. Quantum 16 memberikan hasil terbaik (WT=7.33, TAT=15.00) karena mendekati perilaku FCFS dengan minim context switch, sementara quantum 2 paling buruk (WT=11.00, TAT=18.67) akibat overhead switching yang tinggi. Namun, trade-off-nya adalah quantum besar mengurangi responsivitas sistem untuk proses

interaktif, menjadikan pemilihan quantum optimal sebagai kunci keseimbangan antara efisiensi dan fairness dalam Round Robin.

2.5.3 Fairness

- Algoritma mana yang paling "adil" untuk semua proses?

Round Robin dianggap sebagai algoritma paling adil karena menjamin setiap proses mendapatkan slot waktu CPU yang sama melalui mekanisme time quantum, mencegah proses tertentu mendominasi sumber daya secara berlebihan. Meskipun menghasilkan turnaround time yang tidak selalu optimal, Round Robin memastikan fairness dengan memberikan kesempatan eksekusi yang merata kepada semua proses tanpa memandang burst time atau prioritas, sehingga cocok untuk sistem time-sharing yang membutuhkan responsivitas tinggi dan perlakuan yang setara bagi seluruh pengguna.

- Proses mana yang paling dirugikan di algoritma FCFS?

Dalam algoritma FCFS, proses dengan burst time pendek yang datang setelah proses ber-burst time panjang menjadi paling dirugikan karena mengalami convoy effect, dimana mereka harus menunggu proses sebelumnya selesai terlebih dahulu. Akibatnya, waiting time proses pendek meningkat signifikan meskipun kebutuhan eksekusinya kecil, sementara proses panjang di awal justru mendapatkan layanan langsung tanpa antrian. Ketidakadilan ini terutama terlihat ketika variasi burst time antar proses sangat besar, membuat FCFS tidak suitable untuk lingkungan dengan beban kerja heterogen.

- Apakah ada kemungkinan starvation di SJF?

SJF memiliki kerentanan terhadap starvation dimana proses dengan burst time panjang mungkin tidak pernah dieksekusi jika terus-menerus datang proses baru yang lebih pendek, mengakibatkan indefinite postponement. Meskipun optimal untuk meminimalkan waiting time, sifat greedy SJF dalam memprioritaskan tugas terpendek menciptakan ketidakadilan sistemik terhadap proses komputasi berat. Dalam implementasi praktis, teknik aging biasanya diperlukan untuk mencegah starvation dengan secara gradual meningkatkan prioritas proses yang telah menunggu lama.

2.5.4 Real World Application

- Untuk web server yang melayani banyak request kecil, algoritma mana yang cocok?

Round Robin sangat cocok untuk web server karena karakteristik request HTTP yang umumnya berdurasi pendek dan membutuhkan responsivitas tinggi. Mekanisme time quantum pada Round Robin memastikan fairness dengan mencegah request panjang memonopoli resources, sekaligus menjaga throughput optimal melalui distribusi waktu pemrosesan yang merata. Implementasi ini meminimalkan latency untuk koneksi simultan dan mencegah starvation pada request pendek, sehingga sesuai dengan arsitektur web server modern yang menangani ribuan koneksi konkuren secara efisien.

- Untuk render video (task besar), algoritma mana yang cocok?

FCFS merupakan pilihan ideal untuk task komputasi berat seperti render video karena minim overhead context switch dan mampu memaksimalkan throughput untuk proses berdurasi panjang. Algoritma ini memungkinkan proses rendering menyelesaikan tasknya secara kontinu tanpa interupsi, sehingga mengoptimalkan utilisasi CPU dan meminimalkan wasted cycles akibat switching. Dalam konteks batch processing yang tidak memerlukan interaktivitas, FCFS memberikan efisiensi tertinggi dengan menyelesaikan task besar secara sekuensial hingga tuntas.

- Untuk OS desktop (interactive), algoritma mana yang cocok?

Round Robin dengan quantum optimal paling sesuai untuk OS desktop karena menjamin responsivitas sistem melalui time slicing yang adil bagi semua aplikasi interaktif. Algoritma ini mencegah starvation dengan memberikan jatah waktu reguler kepada setiap proses, sehingga pengguna dapat berinteraksi dengan multiple aplikasi secara simultan tanpa mengalami hang. Kombinasi fairness dan low latency membuat Round Robin ideal untuk lingkungan desktop dimana pengalaman pengguna dan responsivitas menjadi prioritas utama dibanding throughput maksimal.

BAB III

PENUTUP

3.1 KESIMPULAN

Berdasarkan analisis komprehensif terhadap tiga algoritma penjadwalan CPU, dapat disimpulkan bahwa setiap algoritma memiliki domain optimalitasnya masing-masing yang bergantung pada karakteristik spesifik workload dan requirements sistem. FCFS terbukti efektif untuk lingkungan batch processing dengan proses panjang yang dominan, meskipun rentan terhadap convoy effect yang merugikan proses pendek. SJF secara konsisten meminimalkan waiting time namun mengorbankan fairness dengan risiko starvation pada proses komputasi intensif. Round Robin menjamin responsivitas dan equity melalui mekanisme time slicing, meskipun dengan konsekuensi overhead context switch yang signifikan, terutama dengan quantum kecil. Pemilihan algoritma yang tepat harus mempertimbangkan trade-off kompleks antara throughput, latency, fairness, dan overhead sistem, dimana implementasi hybrid atau adaptive scheduling seringkali diperlukan untuk menangani dinamika workload modern yang semakin kompleks dan bervariasi.